```
# Task 1: Data Preparation
!pip install yfinance

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler

# 1. Download Apple (AAPL) stock data
# Added auto_adjust=True and manual column selection to fix the MultiIndex issue
df = yf.download('AAPL', start='2019-01-01', end='2024-01-01', auto_adjust=True)

# 2. Feature Selection: Directly access the 'Close' column
# This handles the case where yfinance returns a multi-level column header
if isinstance(df.columns, pd.MultiIndex):
    data = df['Close']['AAPL'].values.reshape(-1, 1)
else:
    data = df[['Close']].values

# 3. Scaling: Normalizing data to range (0,1)
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# 4. Create Training Dataset (80% training)
training_data_len = int(np.ceil(len(data) * .8))
train_data = scaled_data[0:int(training_data_len), :]

# Prepare x_train and y_train (using 60-day window)
x_train = []
y_train = []

for i in range(60, len(train_data)):
    x_train.append(train_data[i-60:i, 0])
    y_train.append(train_data[i, 0])

x_train, y_train = np.array(x_train), np.array(y_train)

# Reshape data for LSTM [samples, time steps, features]
x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))

print(f"Data Preparation Complete.")
print(f"Total records found: {len(data)}")
print(f"Training shape: {x_train.shape}")
```

```
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (0.2.66)
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.2.2)
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.0.2)
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.32.4)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.0.13)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.9.6)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2025.2)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.4.7)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.0.5)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.13.5)
Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.15.0)
Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (5.29.6)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (15.0.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfinance) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfinance) (4.15.0)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2.0.0)
Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2026.4.22)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (13.9.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3.0->yfinance) (2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3.0->yfinance) (2026.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (3.13)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (2.5.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=2.0.0->curl_cffi>=0.7->yfinance) (3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->curl_cffi>=0.7->yfinance) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->curl_cffi>=0.7->yfinance) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->curl_cffi>=0.7->yfinance) (0.1.2
[**********************100%**********************]  1 of 1 completedData Preparation Complete.
Total records found: 1258
Training shape: (947, 60, 1)
```

```
# Task 2 & 3: Model Development and Training
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

# 1. Construct the RNN-LSTM Model
model = Sequential()

# Adding the first LSTM layer and some Dropout regularisation
model.add(LSTM(units=50, return_sequences=True, input_shape=(x_train.shape[1], 1)))
model.add(Dropout(0.2))
```

```
# Adding a second LSTM layer
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))

# Adding the output layer
model.add(Dense(units=1))

# 2. Optimization: Using Adam optimizer and Mean Squared Error loss
model.compile(optimizer='adam', loss='mean_squared_error')

# 3. Training the model
print("Starting training...")
model.fit(x_train, y_train, batch_size=32, epochs=10)
print("Model training complete.")
```

```
Starting training...
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Wher
  super().__init__(**kwargs)
30/30 ━━━━━━━━━━━━━━━━━━━━ 5s 46ms/step - loss: 0.0381
Epoch 2/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 0.0054
Epoch 3/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - loss: 0.0041
Epoch 4/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 0.0036
Epoch 5/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 56ms/step - loss: 0.0036
Epoch 6/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 55ms/step - loss: 0.0035
Epoch 7/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 0.0033
Epoch 8/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - loss: 0.0035
Epoch 9/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 3s 47ms/step - loss: 0.0027
Epoch 10/10
30/30 ━━━━━━━━━━━━━━━━━━━━ 2s 45ms/step - loss: 0.0029
Model training complete.
```

```
# Task 4: Prediction and Visualization
# 1. Prepare the test data set
# We need the last 60 days of training data + the test data
test_inputs = scaled_data[training_data_len - 60:]
x_test = []
# The actual prices for comparison
y_test = data[training_data_len:]

for i in range(60, len(test_inputs)):
    x_test.append(test_inputs[i-60:i, 0])

x_test = np.array(x_test)
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))

# 2. Get the model's predicted price values
predictions = model.predict(x_test)
predictions = scaler.inverse_transform(predictions) # Undo scaling

# 3. Evaluate Performance using RMSE
# This fulfills the metric requirement in the PDF instructions
rmse = np.sqrt(np.mean((predictions - y_test)**2))
print(f"Root Mean Squared Error (RMSE): {rmse}")

# 4. Visualization
# We recreate a temporary dataframe for plotting since 'data' was a numpy array
train_df = pd.DataFrame(data[:training_data_len], columns=['Close'], index=df.index[:training_data_len])
valid_df = pd.DataFrame(data[training_data_len:], columns=['Close'], index=df.index[training_data_len:])
valid_df['Predictions'] = predictions

plt.figure(figsize=(16,8))
plt.title('Stock Price Prediction using RNN-LSTM (AAPL)')
plt.xlabel('Date', fontsize=18)
plt.ylabel('Close Price USD ($)', fontsize=18)
plt.plot(train_df['Close'])
plt.plot(valid_df[['Close', 'Predictions']])
plt.legend(['Train Data', 'Actual Value', 'Model Predictions'], loc='lower right')
plt.show()
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
Root Mean Squared Error (RMSE): 7.96196011494746
```